

Temat:

Konteneryzacja aplikacji – obraz Docker i uruchomienie w Docker Compose.

Cel ćwiczenia:

Rozszerz aplikację z **Ćwiczeń 4–9** o możliwość uruchamiania jej w kontenerze Docker:

- przygotuj **Dockerfile** budujący obraz aplikacji Spring Boot (Java 21),
 - skonfiguruj **Docker Compose**, który uruchomi aplikację jako usługę awir-app,
 - uruchom i przetestuj aplikację działającą w kontenerze.
- W tym ćwiczeniu nie zmieniamy logiki biznesowej – uczymy się, jak spakować istniejący projekt w obraz kontenera i uruchomić go tak, jak robi się to w środowiskach produkcyjnych.

Niezbędne narzędzia:

1. DE: STS 4 / IntelliJ / VS Code
2. Maven 3.9+
3. JDK 21
4. Spring Boot 3.3.x
5. Docker + Docker Compose zainstalowany lokalnie

Dotychczasowy projekt zawierał:

Masz ukończony projekt z poprzednich ćwiczeń:

- warstwa MVC (/users/**) i REST (/api/users/**),
- Security (Basic Auth dla REST, formLogin + remember-me + ewentualnie OAuth2),
- SOAP Web Service (ćw. 7),
- wysyłanie maili (ćw. 8),
- Spring Integration HTTP gateway (ćw. 9).

Aplikacja działa lokalnie poleceniem:

```
mvn spring-boot:run
```

W tym ćwiczeniu nauczysz się uruchamiać ją tak:

```
docker compose up
```

1. Budoawanie pliku JAR

W pom.xml projektu masz już plugin Spring Boot:

```
<build>
  <plugins>
    <plugin>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-maven-plugin</artifactId>
    </plugin>
  </plugins>
</build>
```

Zbuduj aplikację w trybie jar z poziomu środowiska lub z linii komend:

```
mvn clean package -DskipTests
```

Po udanym buildzie powinien powstać plik JAR w katalogu target/, np.:

- target/awir-0.0.1-SNAPSHOT.jar

2. Tworzenie Dockerfile

W katalogu głównym projektu (tam gdzie pom.xml) utwórz plik tekstowy **Dockerfile**.

Przykładowa zawartość (prosty, jednofazowy build – JAR budujemy wcześniej Mavenem):

```
# Lekka baza z Javą 21
FROM eclipse-temurin:21-jdk-alpine

# Argument z nazwą JAR-a (domyślnie jak z Mavena)
ARG JAR_FILE=target/awir-0.0.1-SNAPSHOT.jar

# Katalog roboczy w kontenerze
WORKDIR /app

# Skopiuj JAR do kontenera
COPY ${JAR_FILE} app.jar

# Zmienna na dodatkowe opcje JVM (opcjonalnie)
ENV JAVA_OPTS=""

# Aplikacja Spring Boot słucha domyślnie na 8080
EXPOSE 8080

# Komenda startowa
ENTRYPOINT ["sh", "-c", "java $JAVA_OPTS -jar app.jar"]
```

3. Budowanie obrazu Dockera

Zbuduj obraz korzystając z Dockerfile (wywołaj poniższą komendę w miejscu gdzie znajduje się Dockerfile):

```
docker build -t awir-app:1.0 . # na końcu ważna kropka!
```

- -t awir-app:1.0 – nazwa i tag obrazu
- . – kontekst builda (bieżący katalog).

Sprawdź, czy obraz istnieje:

```
docker images | grep awir-app
```

Test działania

Możesz testowo uruchomić kontener bez compose:

```
docker run --rm -p 8080:8080 awir-app:1.0
```

Wejdz na <http://localhost:8080> i sprawdź, czy aplikacja działa.

Zatrzymaj kontener Ctrl+C.

4. Utworzenie docker-compose.yml

W katalogu głównym projektu (obok pom.xml i Dockerfile) utwórz plik **docker-compose.yml**:

```
version: "3.8"
```

```
services:
```

```
  awir-app:
    build: .
    container_name: awir-app
    ports:
      - "8080:8080"
```

Opis:

- build: . – Compose sam użyje Dockerfile z bieżącego katalogu,
- ports: "8080:8080" – udostępni aplikację na hoście pod localhost:8080

5. Uruchomienie aplikacji przez Docker Compose

- Upewnij się, że projekt się buduje (mvn clean package -DskipTests).
- Uruchom Compose:
docker compose up -build
- Sprawdź logi kontenera:
docker compose logs -f awir-app
- Sprawdź w przeglądarce działanie aplikacji
- Zatrzymanie kontenera:
docker compose down

6. Zadania do wykonania

- Zbuduj JAR aplikacji (mvn clean package -DskipTests) i przygotuj **Dockerfile** dla obrazu z Javą 21.
- Zbuduj obraz Dockera: docker build -t awir-app:1.0 . i zweryfikuj, że kontener działa (docker run -p 8080:8080 awir-app:1.0).
- Utwórz plik docker-compose.yml z usługą awir-app, która buduje obraz z Dockerfile i wystawia port 8080.
- Uruchom aplikację przez docker compose up i sprawdź dostępność głównych endpointów (MVC, REST).